

Python para Ciência de Dados

Manipulação de Dados com Pandas



O que é o Pandas?



Biblioteca Python

Open-source para análise e manipulação de dados tabulares



Alta Performance

Operações vetorizadas sobre milhões de linhas com eficiência



Formatos

Lê CSV, Excel, JSON, SQL, Parquet e muitos outros formatos



Limpeza

Renomear colunas, filtrar, tratar nulos e transformar dados

Leitura de Arquivos

CSV

- ▶ Formato texto separado por vírgulas
- ▶ Leve e amplamente compatível
- ▶ Ideal para datasets pequenos/médios
- ▶ Função: `pd.read_csv()`

Excel

- ▶ Formato .xlsx do Microsoft Excel
- ▶ Suporta múltiplas abas (sheets)
- ▶ Mais pesado que CSV
- ▶ Função: `pd.read_excel()`

 Dica: ambas as funções retornam um DataFrame — a estrutura central do Pandas.

Leitura de Arquivos — Código

```
import pandas as pd

# Lendo um arquivo CSV
df = pd.read_csv("dados.csv")

# Lendo um arquivo Excel
df = pd.read_excel("planilha.xlsx", sheet_name="Vendas")

# Visualizando as primeiras linhas
print(df.head())
```

O resultado é sempre um DataFrame (df) — uma tabela com linhas e colunas indexadas.

Renomeação de Colunas

Por que renomear?



Colunas com espaços causam erros na sintaxe do Python



Dados importados frequentemente chegam em outro idioma



Nomes muito longos dificultam a leitura do código



Padronizar nomenclatura melhora colaboração em equipes



Método principal: `df.rename(columns={...})`

Renomeação de Colunas — Código

```
# Método 1: renomear colunas específicas
df = df.rename(columns={
    "First Name": "nome",
    "Last Name": "sobrenome",
    "Age": "idade"
})

# Método 2: renomear todas as colunas de uma vez
df.columns = ["nome", "sobrenome", "idade"]

# Verificar resultado
print(df.columns.tolist())
```

Seleção de Linhas e Colunas

Seleção de Colunas

```
df["nome"]
```

Seleciona 1 coluna → Series

```
df[["nome", "idade"]]
```

Seleciona múltiplas → DataFrame

```
df.iloc[:, 0:3]
```

Colunas por posição (0 a 2)

Seleção de Linhas

```
df.iloc[0]
```

Linha pela posição (índice 0)

```
df.loc[df["idade"] > 18]
```

Filtro condicional (booleano)

```
df.head(5)
```

Primeiras 5 linhas

loc vs iloc — Qual usar?

.loc[]

Acesso por rótulo (label)

```
df.loc[2, "nome"]  
# linha com índice 2,  
# coluna chamada "nome"
```

- ✓ Usa nomes reais das colunas
- ✓ Usa valores do índice do df

.iloc[]

Acesso por posição inteira

```
df.iloc[2, 0]  
# 3ª linha (pos 2),  
# 1ª coluna (pos 0)
```

- ✓ Usa posição 0, 1, 2...
- ✓ Funciona como lista Python

Recapitulando...

01

Leitura de Arquivos

```
pd.read_csv("arquivo.csv")
```

```
pd.read_excel("arquivo.xlsx")
```

02

Renomeação de Colunas

```
df.rename(columns={"antigo": "novo"})
```

```
df.columns = ["a", "b", "c"]
```

03

Seleção de Colunas

```
df["coluna"] ou df[["col1", "col2"]]
```

```
df.iloc[:, 0:3] – por posição
```

04

Seleção de Linhas

```
df.loc[df["idade"] > 18] – por rótulo
```

```
df.iloc[0:5] – por posição
```

Exercicio 1

```
# Método 1: renomear colunas específicas
df = df.rename(columns={
    "First Name": "nome",
    "Last Name": "sobrenome",
    "Age": "idade"
})

# Método 2: renomear todas as colunas de uma vez
df.columns = ["nome", "sobrenome", "idade"]

# Verificar resultado
print(df.columns.tolist())
```

Exercicio 2

```
# Método 1: renomear colunas específicas
```

```
df = df.rename(columns={  
    "First Name": "nome",  
    "Last Name": "sobrenome",  
    "Age": "idade"  
})
```

```
# Método 2: renomear todas as colunas de uma vez
```

```
df.columns = ["nome", "sobrenome", "idade"]
```

```
# Verificar resultado
```

```
print(df.columns.tolist())
```



Pronto para codar!

Agora você já sabe como carregar, renomear e selecionar dados com Pandas.

```
pip install pandas
```